

CSR Automation Toolkit — Setup Guide

Prerequisites

- Python 3.8+
 - Gmail account (for email integration)
 - Slack workspace with admin access (for webhook)
 - Google Sheets (for audit logging)
 - OpenAI API key (optional, for LLM-based classification)
-

1. Installation

Install dependencies

```
pip install tkinter requests gspread oauth2client python-dotenv openai
```

On macOS with Homebrew:

```
brew install python-tk@3.11
```

Copy the script

Save `csr_automation_toolkit.py` to your working directory.

2. Environment Configuration

Create a `.env` file in the same directory as the script:

```
touch .env
```

Add your credentials:

```
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/YOUR/WEBHOOK/URL
EMAIL_ADDRESS=your-email@gmail.com
EMAIL_PASSWORD=your-app-password
GSHEETS_ID=your-google-sheet-id
GSHEETS_CREDENTIALS={"type":"service_account","project_id":"..."}
OPENAI_API_KEY=sk-your-openai-key
```

2a. Generate Gmail App Password

1. Go to myaccount.google.com/security
2. Enable 2-Step Verification
3. Go to "App passwords" → Select Mail and Windows Device → Generate
4. Copy the 16-character password into `EMAIL_PASSWORD`

2b. Set up Slack Webhook

1. Go to slack.com/apps → Search "Incoming Webhooks"
2. Click "Add to Slack" → Choose your workspace and channel
3. Copy the Webhook URL into `SLACK_WEBHOOK_URL`

2c. Set up Google Sheets Integration

1. Create a new Google Sheet with columns: `Timestamp` | `Customer Email` | `Query` | `Response` | `Follow-Up`
2. Go to [Google Cloud Console](https://console.cloud.google.com) → Create a new project
3. Enable "Google Sheets API" and "Google Drive API"
4. Create a Service Account and generate a JSON key
5. Share your Google Sheet with the service account email
6. Copy the JSON key content into `GSHEETS_CREDENTIALS` (as single-line JSON)
7. Copy the Sheet ID (from the URL) into `GSHEETS_ID`

2d. Set up OpenAI (Optional)

Get your API key from platform.openai.com/api-keys

Add to `.env` :

```
OPENAI_API_KEY=sk-...
```

If not configured, the toolkit falls back to keyword matching.

3. Customize Standard Responses

Edit the `STANDARD_RESPONSES` list in `csr_automation_toolkit.py`:

```
STANDARD_RESPONSES = [  
    (["keyword1", "keyword2"], "response_id", "Your response text here.", False),  
    # follow_up flag: True if this type of query needs escalation/follow-up  
]
```

Example:

```
(["billing", "invoice", "payment"], "response_billing",  
 "For billing questions, please provide your account number so I can assist.", Tr
```

4. Run the Application

```
python csr_automation_toolkit.py
```

The GUI will open. You can now:

- Paste customer queries and get auto-responses
 - Check emails periodically (auto-checks every 5 minutes)
 - All interactions are logged to Google Sheets
 - Follow-up items are tagged in Slack as `[FOLLOW-UP NEEDED]`
-

5. Deployment Options

Local Desktop

Run on your machine. Emails check every 5 minutes when the app is open.

Scheduled Server

Run on a server with cron or systemd timer:

```
# Check emails every hour
0 * * * * /usr/bin/python3 /path/to/csr_automation_toolkit.py --email-only
```

Docker Container

Create a `Dockerfile`:

```
FROM python:3.11-slim
WORKDIR /app
COPY csr_automation_toolkit.py .
COPY requirements.txt .
RUN pip install -r requirements.txt
CMD ["python", "csr_automation_toolkit.py"]
```

Build and run:

```
docker build -t csr-toolkit .
docker run -d --env-file .env csr-toolkit
```

6. Troubleshooting

“Failed to send Slack alert”

- Verify `SLACK_WEBHOOK_URL` is correct and active
- Check network connectivity

“Failed to log to Sheets”

- Confirm service account email is shared on the Google Sheet
- Verify `GSHEETS_ID` and `GSHEETS_CREDENTIALS` are correct

“Failed to check emails”

- Use an app-specific Gmail password (not your regular password)
- Ensure IMAP is enabled in Gmail settings (Settings → Forwarding and POP/IMAP)

“AI classification error”

- Verify OPENAI_API_KEY is valid
 - Check OpenAI account has available credits
-

7. Performance & Monitoring

Monitor your Google Sheets audit log:

- **Timestamp:** When the query was processed
- **Customer Email:** Who sent it (if email integration)
- **Query:** What they asked
- **Response:** What you sent
- **Follow-Up:** Whether it needs escalation

Use Slack notifications to catch `[FOLLOW-UP NEEDED]` items in real time.

8. Next Steps

1. Customize STANDARD_RESPONSES for your use cases
 2. Train your team on the GUI
 3. Connect Slack alerts to Asana (via Zapier) for automatic task creation
 4. Review audit logs weekly for response effectiveness
 5. Iterate on response templates based on customer feedback
-

Questions? Check the GitHub README or reach out.